ReconX

Build Plan: Cross-Platform Passive Recon & Attack-Surface Intel CLI

Scope: Windows + Kali Linux CLI. Input = domain / IP / CIDR / org. Output = correlated, deduplicated, source-attributed OSINT on the target's external attack surface.

Mentor framing: This is a real, legitimate recon/OSINT tool used in authorized pentest engagements (same category as Amass, Subfinder, theHarvester). It only aggregates *publicly available* data — no exploitation, no unauthorized access. Keep it that way: passive by default, active scanning gated behind explicit flags + written authorization.

How to use this plan: Don't build everything at once. Follow the phases top to bottom. Each phase has a ■/■/■ priority so you know what's core vs. what can wait. Ship a working v1 with 2–3 providers before chasing coverage.

1. Goal, Non-Goals, and Ground Rules

Goal

A single binary (Windows/Linux) that takes a domain, IP, CIDR, or org name and returns a normalized, deduplicated, confidence-scored map of the target's externally visible infrastructure — built from many passive sources instead of one.

Non-Goals (v1)

- No active exploitation, brute-forcing, or credential testing.
- No automatic requests to discovered endpoints (that's a separate active module, opt-in only).
- No scraping sources whose ToS forbid automated access — check each provider's terms before adding it.

Ground Rules

- Only ever run this against assets you own or have written authorization to test.
- Passive vs. active must be an explicit, separate flag (`--passive` default, `--active` requires confirmation).
- Every finding carries: source, first_seen/last_seen, confidence. No orphan facts.

2. What To Learn/Build First — Priority Map

Building this tool touches Go, DNS, APIs, and recon methodology simultaneously. Don't context-switch randomly — use this order.

Priority	Track	Why
1	Go fundamentals, interfaces, goroutines, HTTP client	You cannot build the provider architecture without this. Do this
2	DNS deep dive	This is the core data your tool produces. If you don't understa
3	Provider interface + one working provider (crt.sh) end-to-end	Proves the architecture works before you scale to 10 provider
4	Config + API key management, concurrency patterns	Needed once you have 3+ providers hitting real APIs.
5	SQLite storage	Needed once data volume makes flat JSON unmanageable.
6	Output formats (HTML report)	Needed for deliverable polish, not for core functionality.
7	Graph analysis (Neo4j)	Interesting later. Will distract you from shipping v1 now.
8	Active/HTTP probing	Deliberately deferred — separate authorization and safety sur

3. Architecture (What You're Actually Building)

```
CLI (cobra)
-> Target Parser (domain / IP / CIDR / org -> typed Target)
-> Orchestrator (fan-out to providers, respects --passive/--active)
-> Provider Layer (each source implements one Go interface)
-> Normalizer (raw provider JSON -> common asset schema)
-> Deduplicator (same asset from N sources -> 1 record + source list)
-> Correlator (domain -> subdomain -> IP -> ASN -> org graph)
-> Storage (SQLite)
-> Reporter (terminal / JSON / CSV / HTML / Markdown)
```

Provider Interface (Go, conceptual)

```
type Provider interface {
Name() string
Supports(t Target) bool
Collect(ctx context.Context, t Target) ([]RawResult, error)
RateLimit() time.Duration
}
```

Every new source (crt.sh, VirusTotal, Shodan, Censys...) is just a new struct implementing this. The orchestrator never needs to change when you add a provider — that's the whole point of the abstraction.

Core Data Model (the thing everything normalizes into)

```
{
"asset": "api.example.com",
"type": "subdomain",
"sources": ["crtsh", "virustotal"],
"first_seen": "2024-01-10",
"last_seen": "2026-09-01",
"confidence": 0.92,
"relationships": {
"resolves_to": ["203.0.113.10"],
"asn": "AS64500",
"org": "Example Corp"
}
}
```

4. Tech Stack Decision

Layer	Choice	Why
Language	Go 1.22+	Cross-compiles to single binaries for Windows/Linux/ARM with zero runtime
CLI framework	cobra + viper	Industry standard for exactly this kind of tool (used by kubectl, hugo, etc.). V
HTTP	net/http + resty (optional)	resty simplifies retries/timeouts; stdlib is enough for v1.
DNS	miekg/dns	The de-facto Go DNS library; used internally by dnsx/subfinder.
Storage	SQLite via mattn/go-sqlite3 or modernc.org/sqlite (pure Go, no CGO — avoids C toolchain pain when cross-compiling for Windows.	
Config	YAML (gopkg.in/yaml.v3) + env var interpolation	Matches config.yaml example in your spec.
Reporting	html/template + Go Markdown lib (yuin/goldmark for parsing, hand-rolled for generating)	Keeps binary lean and dependency-light.

5. Phased Build Roadmap

Each phase should be a working, testable milestone — not a half-finished sprawl. Don't start Phase N+1 until Phase N runs end-to-end.

Phase 0 — Skeleton CLI ■ CORE

Goal: A binary that parses args and prints something, cross-compiled for both OSes.

- Set up Go module, cobra CLI skeleton: `reconx --help`, `reconx --version`.
- Target parser: detect domain vs IPv4 vs IPv6 vs CIDR vs org string (regex + net.ParseIP + net.ParseCIDR).
- Structured logging (log/slog) and consistent error wrapping (fmt.Errorf with %w).
- GitHub Actions workflow that cross-compiles for windows/amd64, linux/amd64, linux/arm64.

Done when: `reconx scan --domain example.com` runs on both a Windows machine and Kali and prints a stub summary.

Phase 1 — Domain Intelligence Core ■ CORE

Goal: Real DNS + WHOIS + Certificate Transparency for one domain, no correlation yet.

- WHOIS: shell out to system whois or use a Go WHOIS client; parse registrar/dates/nameservers.
- DNS: implement A/AAAA/CNAME/MX/NS/TXT/SOA/CAA lookups via miekg/dns.
- crt.sh provider: query `https://crt.sh/?q=%25.example.com&output=json`, parse SANs into subdomain list.
- Print raw findings to terminal in a readable table (no dedup/correlation yet — that's Phase 3).

Done when: For a real domain, you get accurate DNS records + a subdomain list pulled from crt.sh, verified by hand against crt.sh's website.

Phase 2 — Provider Framework + 2nd/3rd Provider ■ CORE

Goal: Prove the plugin architecture by adding sources without touching the core.

- Define the Provider interface exactly as in Section 3.
- Refactor crt.sh into a Provider implementation.
- Add VirusTotal subdomain provider (free-tier API, needs API key in config.yaml).
- Add one Passive-DNS-style provider (e.g. HackerTarget or RapidDNS free endpoints — check ToS first).
- Orchestrator fans out providers concurrently (goroutines + errgroup), respects per-provider RateLimit().

Done when: Adding a 4th provider requires writing one new file and one line of registration — zero changes to orchestrator/CLI code.

Phase 3 — Normalization, Dedup, Correlation ■ SECONDARY

Goal: Turn 5 providers' worth of noisy overlapping output into one clean asset list.

- Normalizer: map every provider's raw shape into the common asset schema (Section 3).
- Dedup: key by (asset value + type); merge source lists instead of printing duplicates.
- Confidence score: simple v1 formula — e.g. 0.5 base + 0.1 per additional independent source, capped at 0.98.
- Correlation: build the domain -> subdomain -> IP -> ASN -> org chain as an in-memory graph.

Done when: Running scan against a domain with subdomains found by 3 different providers shows ONE entry with all 3 sources listed, not 3 rows.

Phase 4 — IP / ASN Intelligence ■ SECONDARY

Goal: Resolve every discovered hostname and enrich the resulting IPs.

- Bulk-resolve all discovered hostnames to A/AAAA records.
- ASN/org lookup via a free IP intel source (e.g. Team Cymru whois, ipinfo.io free tier, or BGPView API).
- Reverse DNS (PTR) for discovered IPs.
- Attach ASN/org/hosting-provider metadata to each IP node in the correlation graph.

Done when: For a discovered subdomain, the tool shows: hostname -> IP -> ASN -> org -> hosting/cloud provider, in one chain.

Phase 5 — Storage (SQLite) + Historical Sources ■ SECONDARY

Goal: Persist results across runs; add Wayback/Common Crawl for historical URLs.

- SQLite schema: targets, assets, sources, relationships tables.
- Store each scan as a run, so you can diff two scans of the same target over time (attack-surface drift).
- Wayback Machine CDX API provider for historical URLs/subdomains.
- Common Crawl index provider (optional, heavier).

Done when: Running the same scan twice, a week apart, produces a diff: 'new subdomain X appeared since last scan.'

Phase 6 — Reporting ■ SECONDARY

Goal: Professional, copy-ready output for a pentest report.

- Terminal summary dashboard (counts by category, high-confidence assets highlighted).
- JSON/JSONL export matching the schema in Section 3 exactly.
- HTML report (Go html/template) with executive summary + evidence tables.
- Markdown report for quick paste into engagement notes.

Done when: You can hand the HTML report to a client-facing pentester and they don't need to touch the CLI.

Phase 7 — Cross-Platform Packaging & Release ■ SECONDARY

Goal: Ship it as real binaries, not 'clone the repo and go run'.

- GoReleaser config for windows/amd64, windows/arm64, linux/amd64, linux/arm64.
- config.yaml.example with all providers documented, API keys via env var interpolation only.
- README with install + quickstart, matching the CLI examples in Section 6.

Done when: A stranger can download reconx-windows-amd64.exe or reconx-linux-amd64, drop a config.yaml, and run a scan with zero build tools installed.

Phase 8 — Active Module (explicit opt-in, deferred) ■ LATER/OPTIONAL

Goal: Only after v1 passive tool is solid and you have a real authorized target to test against.

- httpx-style HTTP probing of discovered hosts (title, status, tech headers) — behind `--active`.
- Mandatory authorization banner/confirmation before any active module runs.
- Rate-limited, polite by default; never port-scan without explicit user confirmation of scope.

Done when: Not a v1 concern — revisit once passive tool is stable and you're doing real authorized engagements.

6. Target CLI Design

```
reconx scan --domain example.com --passive --output ./results
reconx subdomains --domain example.com
reconx dns --domain example.com
reconx certificates --domain example.com
reconx ip --target 8.8.8.8
reconx history --domain example.com
reconx scan --domain example.com --html report.html --json results.json
```

Keep every subcommand independently runnable (not just through `scan`) — this makes testing and debugging individual providers far easier.

7. Project Layout

```
reconx/
cmd/reconx/main.go
internal/
cli/ cobra commands
target/ input parsing (domain/ip/cidr/org)
orchestrator/ fan-out + rate limiting
providers/ one file per source (crtsh.go, virustotal.go, ...)
normalize/ raw -> common schema
dedup/ dedup + confidence scoring
correlate/ relationship graph
storage/ sqlite
report/ json/csv/html/markdown
pkg/models/ shared structs (Target, Asset, Relationship)
configs/config.yaml.example
tests/
go.mod
```

8. Testing Strategy

- Unit test the target parser against edge cases: IPv6, CIDR, internationalized domains, malformed input.
- Mock providers (implement the same interface, return canned data) so orchestrator/dedup/correlation logic can be tested without hitting real APIs or burning rate limits.
- Golden-file tests for normalization: raw provider JSON in -> expected common-schema JSON out.
- Test failure modes explicitly: provider timeout, invalid API key, malformed API response, rate-limit 429 — the tool should degrade gracefully (skip that provider, log a warning, keep going), never crash the whole scan.
- Only ever integration-test against domains/IPs you own or are authorized to query — even passive OSINT queries should go against your own lab domains during development.

9. Immediate Next Step

Don't start with providers. Start with **Phase 0**: a Go CLI skeleton that cross-compiles for Windows and Linux and correctly classifies input as domain/IP/CIDR/org. That's a self-contained, completable task you can finish this week and verify end-to-end before any recon logic exists.

Prepared as a working build plan — not a final spec. Revisit priorities after each phase; cut scope from ■/■ items before cutting scope from ■ items.